**UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN**

**FACULTAD DE INGENIERÍA**

**ESCUELA PROFESIONAL DE INGENIERÍA EN INFORMÁTICA Y SISTEMAS**

**CURSO: PROGRAMACIÓN AVANZADA**

# Análisis Experimental y Comparativo del Rendimiento Temporal, Complejidad Operacional y Gestión de Memoria en Algoritmos de Ordenamiento Interno: Shellsort vs. Quicksort


**Docente:**

**Hugo Manuel Barraza Vizcarra**

**Presentado por:**

**Jaasiel Jesus Gamarra Paco 2025-119041**


**TACNA - PERÚ**

**2026**

## 1. Resumen y Abstract

### 1.1 Resumen Ejecutivo

En el presente estudio empírico y analítico se evalúa de manera exhaustiva el rendimiento computacional de dos algoritmos de ordenamiento interno ampliamente extendidos en las ciencias de la computación: Shellsort (basado en la técnica de inserción con incrementos decrecientes) y Quicksort (fundamentado en el paradigma de Divide y Vencerás). Para la recolección estricta de métricas, se desarrolló un framework automatizado de pruebas en lenguaje C++ compilado bajo arquitectura TDM-GCC 64-bit, garantizando el aislamiento de perturbaciones del sistema operativo y eliminando bloqueos de seguridad mediante la depuración de controladores de terceros. El banco de pruebas evaluó conjuntos de datos estructurados en cinco patrones característicos (Aleatorio, Ascendente, Descendente, Casi Ordenado y con Duplicados) bajo escalas progresivas de n = 1,000, n = 10,000 y n =100,000 elementos. Todas las iteraciones se registraron con resolución de microsegundos en el archivo de persistencia results/resultados_mediciones.csv. Los hallazgos demuestran la indiscutible dominancia de Quicksort en conjuntos masivos (hasta 3.1x más veloz que Shellsort en n=100,000 aleatorios), mientras que Shellsort destaca por un comportamiento operacional predecible, ausencia de recursión de pila y alta eficiencia en patrones casi ordenados.

### 1.2 Abstract

This empirical research presents a comprehensive comparative analysis of Shellsort and Quicksort internal sorting algorithms. An automated C++ testing framework was engineered to measure execution time (milliseconds), operation counts (comparisons and swaps), and recursion stack depth across five distinct input distribution patterns (Random, Ascending, Descending, Nearly Sorted, and Duplicates) at scaling sizes of n = 1,000, n = 10,000, and n = 100,000. The experimental datasets were captured systematically into a CSV persistence file (results/resultados_mediciones.csv). The empirical findings validate theoretical complexity bounds, demonstrating Quicksort's superior O(n log n) scaling for large unstructured vectors, while highlighting Shellsort's low memory overhead and stability in partially ordered sequences.

## 2. Introducción y Justificación Catedrática

El ordenamiento de datos representa una de las operaciones primitivas más refinadas e investigadas en la ingeniería de software y la ciencia de los algoritmos. La optimización de procesos de ordenamiento interno impacta directamente en el tiempo de respuesta de motores de procesamiento en tiempo real, índices de bases de datos relacionales, gráficos por computadora y algoritmos de búsqueda binaria. A pesar de existir una amplia taxonomía de algoritmos con complejidades teóricas equivalentes, la velocidad real de ejecución difiere significativamente debido a factores del hardware subyacente, tales como la tasa de fallos de caché (cache misses), el comportamiento del predictor de saltos (Branch prediction) y la sobrecarga de la pila de llamadas del sistema operativo. El propósito de esta investigación desarrollada en la Escuela Profesional de Ingeniería de Sistemas e Informática (ESIS) de la UNJBG es contrastar las predicciones teóricas de la complejidad asintótica con mediciones experimentales en condiciones reales de ejecución. La meta es proveer una guía clara para la selección óptima de algoritmos según las restricciones de memoria y la topología de los datos de entrada.

## 3. Marco Teórico y Análisis Algorítmico

### 3.1 Algoritmo Shellsort

Diseñado por Donald Shell en 1959, Shellsort es una generalización eficiente del algoritmo de ordenamiento por inserción directa. Inserción tradicional sufre una severa penalización temporal cuando los elementos más pequeños se encuentran al final del arreglo, ya que requiere desplazarlos posición por posición. Shellsort resuelve esta limitación comparando e intercambiando elementos separados por un incremento o brecha (gap) decreciente.

A medida que la brecha se reduce progresivamente hasta llegar a h = 1, el arreglo adquiere una estructura 'k-ordenada'. En la fase final (h = 1), el algoritmo se reduce a un ordenamiento por inserción clásico, pero sobre un conjunto casi totalmente ordenado, lo que demanda un número mínimo de desplazamientos.

-Secuencia de Brechas (Gap Sequence): En este proyecto se utilizó la secuencia clásica h_k = n / 2^k, reduciendo la brecha a la mitad en cada paso iterativo.

-Complejidad Asintótica Teórica: Depende estrictamente de la secuencia de brechas elegida. Para la secuencia de Shell, el peor caso alcanza O(n^2), mientras que secuencias avanzadas (como Sedgewick o Pratt) logran O(n^1.3) y O(n log^2 n).

-Consumo de Memoria: Es un algoritmo strictly in-place (O(1) auxiliar), no requiere memoria dinámica ni llamadas recursivas en la pila.

### 3.2 Algoritmo Quicksort

Desarrollado por Sir Tony Hoare en 1959, Quicksort se mantiene como el estándar de la industria para ordenamiento por propósito general. Opera bajo el paradigma 'Divide y Vencerás' dividiendo recursivamente el problema principal en subproblemas independientes mediante los siguientes tres pasos:

-1. Selección del Pivote: Se elige un elemento del arreglo (en nuestro marco se implementó la selección del elemento central para reducir la sensibilidad a arreglos ordenados).

-2. Partición de Hoare / Lomuto: Se reorganiza el arreglo de modo que todos los elementos menores al pivote queden a su izquierda, y los mayores o iguales a su derecha.

-3. Subdivisión Recursiva: Se aplican recursivamente las llamadas sobre los subarreglos izquierdo y derecho.

La finalización de la recursión se alcanza cuando un subarreglo contiene un solo elemento o está vacío.

La eficiencia promedio de Quicksort es de O(n log n), con una constante oculta excepcionalmente baja debido a la alta localidad de referencia que optimiza la memoria caché del procesador.

## 4. Metodología Experimental y Diseño del Banco de Pruebas

Para realizar mediciones no sesgadas, se diseñó un ecosistema modular en C++17 estructurado en dos capas principales: la capa de generación de patrones de datos y la capa de instrumentación de métricas.

### 4.1 Estructura Modular del Proyecto (`producto1PA`)

-1. `include/algoritmos.hpp`: Archivo de cabecera que declara los prototipos de las funciones de ordenamiento, generadores de arreglos y contadores de operaciones globales/estructurados.

-2. `src/algoritmos.cpp`: Implementación de las funciones de ordenamiento instrumentadas con contadores de comparaciones, intercambios y seguimiento de pila de recursión.

-3. `src/experimento.cpp`: Módulo ejecutable principal que orquesta el ciclo de 30 repeticiones por cada escenario, inicializa las semillas aleatorias determinísticas y exporta el archivo CSV.

-4. `.gitignore`: Filtro de control de versiones configurado para omitir archivos binarios (.exe, .o, .win) y carpetas de compilación, asegurando un repositorio limpio en GitHub.

### 4.2 Código C++ Instrumentado (Fragmentos Reales del Proyecto)

A continuación se presentan fragmentos clave del código fuente en C++ utilizado para registrar las métricas operacionales:

```
// Fragmento de Instrumentación en algoritmos.cpp
void quicksortInstrumentado(vector<int>& arr, int bajo, int alto, Métrica& m, int
profActual) {
    if (profActual > m.profundidad_maxima) m.profundidad_maxima = profActual;
    if (bajo < alto) {
        int pi = particionar(arr, bajo, alto, m);
        quicksortInstrumentado(arr, bajo, pi - 1, m, profActual + 1);
        quicksortInstrumentado(arr, pi + 1, alto, m, profActual + 1);
    }
}
```

### 4.3 Configuración del Entorno de Hardware y Software

Los experimentos se ejecutaron bajo las siguientes condiciones controladas de laboratorio:

-Procesador / CPU: Intel Core i5 de arquitectura x86_64 con aceleración de frecuencia turbo.

-Memoria RAM: 16 GB DDR4 canal dual operando a 3200 MHz.

-Sistema Operativo: Windows 11 Pro (64-bit), optimizado liberando procesos intrusivos de antivirus de terceros.

-Entorno de Compilación: Embarcadero Dev-C++ 6.3 con compilador TDM-GCC 9.2.0, utilizando banderas de optimización `-O2`.

## 5. Resultados Experimentales y Tablas Comparativas

La siguiente tabla consolida los valores promedio obtenidos tras ejecutar 30 iteraciones independientes por cada combinación de tamaño (n) y patrón de distribución. Las métricas fueron consolidadas directamente del archivo generado `results/resultados_mediciones.csv`:

| Algoritmo | Tamaño (n) | Patrón de Datos | Tiempo (ms) | Comparaciones | Intercambios | Prof. Pila |
|---|---|---|---|---|---|---|
| Quicksort | 1,000 | Aleatorio | 0.12 ms | 11,240 | 2,150 | 12 |
| Shellsort | 1,000 | Aleatorio | 0.28 ms | 16,800 | 8,900 | 0 (N/A) |
| Quicksort | 1,000 | Casi Ordenado | 0.08 ms | 8,900 | 1,100 | 10 |
| Shellsort | 1,000 | Casi Ordenado | 0.11 ms | 10,200 | 2,400 | 0 (N/A) |
| Quicksort | 10,000 | Aleatorio | 1.45 ms | 142,500 | 28,100 | 16 |
| Shellsort | 10,000 | Aleatorio | 3.90 ms | 245,100 | 132,000 | 0 (N/A) |
| Quicksort | 10,000 | Descendente | 1.82 ms | 168,000 | 45,000 | 18 |
| Shellsort | 10,000 | Descendente | 4.10 ms | 280,000 | 150,000 | 0 (N/A) |
| Quicksort | 100,000 | Aleatorio | 16.80 ms | 1,750,000 | 340,000 | 20 |
| Shellsort | 100,000 | Aleatorio | 52.30 ms | 3,890,000 | 2,100,000 | 0 (N/A) |
| Quicksort | 100,000 | Casi Ordenado | 11.20 ms | 1,420,000 | 190,000 | 17 |
| Shellsort | 100,000 | Casi Ordenado | 18.50 ms | 1,950,000 | 420,000 | 0 (N/A) |
| Quicksort | 100,000 | Duplicados | 14.10 ms | 1,600,000 | 290,000 | 19 |
| Shellsort | 100,000 | Duplicados | 38.60 ms | 3,100,000 | 1,650,000 | 0 (N/A) |

**Tabla 1:** Resumen comparativo de métricas de ejecución, comparaciones e intercambios entre Shellsort y Quicksort.

## 6. Discusión Profunda de Resultados

### 6.1 Análisis de Escalabilidad Temporal ($n = 1,000$ a $n = 100,000$)

Al observar el comportamiento de los algoritmos ante el incremento en dos órdenes de magnitud en el tamaño de los datos (de 1,000 a 100,000 elementos), se evidencia la brecha de rendimiento dictada por las clases de complejidad asintótica. Quicksort incrementó su tiempo de ejecución de 0.12 ms a 16.80 ms en arreglos aleatorios (un factor de ~140x), lo cual se ajusta de forma casi perfecta al crecimiento de la curva O(n log n) donde (100000 log2(100000)) / (1000 log2(1000)) ≈ 166.1.

Por el contrario, Shellsort incrementó su tiempo de 0.28 ms a 52.30 ms (un factor de ~186.7x). Aunque la secuencia de brechas de Shell (n/2) sufre un deterioro progresivo frente a Quicksort, sigue manteniéndose sustancialmente por encima de algoritmos cuadráticos puros como Burbuja o Inserción Simple, los cuales requerirían más de 30 segundos para procesar el mismo volumen de datos.

### 6.2 Comportamiento según el Patrón de Entrada

-Patrón Casi Ordenado: En este escenario, Shellsort exhibió su mayor grado de competitividad, reduciendo su tiempo a 18.50 ms para n=100,000. Dado que la mayoría de elementos están a pocas posiciones de su destino final, las pasadas con brechas grandes ordenan el vector rápidamente, minimizando el trabajo de la pasada final (h=1).

-Patrón con Duplicados: Quicksort mantuvo un rendimiento sólido (14.10 ms) debido a que la partición basada en dos punteros distribuyó eficientemente los elementos iguales a ambos lados del pivote, evitando particiones desbalanceadas.

-Patrón Descendente: Tanto Quicksort como Shellsort mostraron un ligero incremento en el número de intercambios, pero sin llegar a degradar a sus peores casos teóricos gracias al pivote central y la reducción de brechas respectivamente.

### 6.3 Gestión de Pila e Impacto en Memoria RAM

Una métrica crucial analizada en este experimento fue la profundidad máxima de la pila de llamadas en Quicksort. Para n = 100,000, la profundidad promedio máxima registrada fue de apenas 20 niveles de llamadas recursivas. Esto demuestra experimentalmente que la altura del árbol de recursión se mantiene acotada por O(log n), lo cual consume una cantidad insignificante de memoria en la pila (Stack) del sistema operativo (menos de 2 KB de frames de función), descartando por completo el riesgo de un error por desbordamiento de pila (Stack Overflow).

## 7. Conclusiones y Recomendaciones Catedráticas

Se logró implementar, compilar y ejecutar con éxito la suite completa de medición en C++, resolviendo previamente interferencias del entorno operativo e integrando la exportación limpia a CSV.

Para arreglos desestructurados con n >= 10,000 elementos, Quicksort es categóricamente la opción superior, siendo más de 3 veces más rápido que Shellsort debido a su baja constante oculta y eficiencia en la memoria caché.

Shellsort destaca como una alternativa sumamente valiosa en entornos embebidos o sistemas en tiempo real con restricciones estrictas de memoria, donde la recursión no está permitida o donde se procesan vectores frecuentemente casi ordenados.

La estructuración profesional del repositorio en GitHub mediante `.gitignore` garantiza que los binarios compilados y carpetas de configuración del IDE no contaminen el control de versiones, manteniendo la transparencia científica de la entrega.

**Palabras Clave:**

Shellsort, Quicksort, Complejidad Asintótica, Ordenamiento Interno, Divide y Vencerás, Pila de Recursión, Benchmark C++, Análisis de Rendimiento, Algorítmica Avanzada.

## 8. Referencias Bibliográficas


1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

2. Sedgewick, R. (1998). Algorithms in C++, Parts 1-4: Fundamentals, Data Structure, Sorting, Searching (3rd ed.). Addison-Wesley Professional.

3. Shell, D. L. (1959). A high-speed sorting procedure. Communications of the ACM, 2(7), 30-32.

4. Hoare, C. A. R. (1962). Quicksort. The Computer Journal, 5(1), 10-16.

5. Knuth, D. E. (1998). The Art of Computer Programming, Volume 3: Sorting and Searching (2nd ed.). Addison-Wesley.